

CyberShield AI

Complete Beginner Guide for the Integrated Digital Rights, Safety and Justice Support System

This guide explains the project as if the reader has no background in programming, machine learning, cybersecurity, or legal technology. It explains what the system does, why each folder exists, what every major script means, how to run it, how to upload it to GitHub, and how to explain it in an interview.

Safety note: This project is a safe technical prototype. It does not provide legal advice, does not submit real FIRs, does not identify real offenders, and must not be used with real victim data without expert legal, privacy, security, and ethics review.

Table of Contents

1. What this project is
2. The real-world problem in simple words
3. The project workflow
4. Folder-by-folder explanation
5. Source code explanation
6. Script-by-script explanation
7. API and dashboard explanation
8. Evidence vault and chain-of-custody
9. Monitoring, audit, and privacy
10. How to run the project locally
11. How to upload to GitHub
12. How to add it to resume
13. How to rebuild a similar project from scratch

1. What this project is

CyberShield AI is a demonstration of how AI can support cybercrime reporting and digital-rights support in a safer, more structured way. A user or support worker enters a cyber incident report. The system classifies the case, scores the urgency, retrieves relevant guidance from a small knowledge base, drafts a complaint-style document, stores evidence hashes, detects repeat anonymous patterns, and shows monitoring information in a dashboard.

In plain language: it turns an unstructured story such as "someone is threatening to leak my private photo" into a structured support package that a human reviewer can understand and improve.

The most important idea is not "AI replaces lawyers or police." The important idea is "AI organizes information, preserves evidence integrity, and helps human reviewers act faster and more consistently."

2. The real-world problem in simple words

Cyber harassment, hacking, blackmail, impersonation, doxxing, and online financial fraud often create fear, confusion, and evidence chaos. Victims may not know what to save, what to write, where to report, or how to explain the situation clearly. Screenshots and chat logs may be scattered across devices. Multiple victims may be targeted by the same anonymous account, but the pattern may remain hidden.

This project addresses that gap as a safe demo. It helps organize victim statements, produce a structured draft, hash evidence, and monitor anonymized repeat patterns. It intentionally avoids real submission, real identity exposure, and final legal conclusions.

3. End-to-end project workflow

Step	What happens	Simple meaning
1	User submits incident report	The system receives the victim statement and basic context.
2	Sensitive text redaction	Emails, phone numbers, and URLs are hidden before further processing.
3	Case classification	The system labels the report, such as blackmail or hacking.
4	Risk scoring	The system estimates whether the case should be standard, medium, or urgent.
5	RAG guidance retrieval	The system searches the local knowledge base for relevant support guidance.
6	Complaint draft generation	The system creates a structured complaint/FIR-style draft for human review.
7	Evidence hashing	Evidence text is hashed and linked into a chain-of-custody ledger.
8	Repeat-pattern tracking	Suspect identifiers are anonymized and checked for repeated patterns.
9	Dashboard and audit logs	Cases, evidence, feedback, and patterns can be reviewed.

4. Folder-by-folder explanation

Folder/File	Purpose
README.md	Main project instructions, quick start, API summary, and safety notes.
ARCHITECTURE.md	Explains how the system components connect together.
SYSTEM_CARD.md	Explains intended use, non-intended use, limitations, and safety design.
ETHICS_AND_SAFETY.md	Explains privacy, human review, and responsible deployment principles.
requirements.txt	List of Python packages needed for the full project.
requirements-minimal.txt	Smaller package list used by GitHub Actions tests.
src/cybershield_ai/	Main application source code. This is where the real system logic lives.
scripts/	Command-line tools for demo, batch triage, monitoring, and case export.
dashboards/	Streamlit dashboard used to view system metrics.
data/demo/	Synthetic demo data and local legal/support knowledge base.
artifacts/	Generated outputs such as demo results, SQLite DB, and reports. Not uploaded with real data.
tests/	Automated tests proving that major components work.
.github/workflows/ci.yml	GitHub Actions CI file that runs tests after each push.
Dockerfile and docker-compose.yml	Files for containerized deployment.

5. Source code explanation

File	What it does
core/models.py	Defines structured input and output objects using Pydantic. These are the shapes of reports, triage results, and evidence items.
core/security.py	Redacts obvious emails, phone numbers, and URLs. It also hashes suspect identifiers so the system can track them without exposing sensitive information.
core/database.py	Creates and manages the SQLite database for cases, evidence, feedback, offender patterns, and offender graphs.
services/case_classifier.py	Classifies incidents using explainable keyword logic. It returns category, confidence, matched terms, and a list of related evidence items.
services/risk_scorer.py	Assigns a risk score based on category, urgent words, immediate danger, evidence count, and repeat patterns.
services/legal_rag.py	Implements a small RAG-style retrieval engine using TF-IDF over a local markdown knowledge base to generate legal advice.
services/fir_generator.py	Creates a structured complaint/FIR-style draft for human review.
services/evidence_vault.py	Creates SHA-256 hashes and chain hashes for evidence integrity.
services/offender_graph.py	Anonymizes suspect identifiers and updates repeat-pattern counts.
services/awareness.py	Returns safety tips based on the incident category.
services/victim_advocate.py	The orchestrator. It calls all other services and produces the final triage result.
api/main.py	FastAPI app exposing endpoints for health, triage, evidence, cases, patterns, feedback, and dashboards.
monitoring/pattern_monitor.py	Builds pattern reports from the database and writes JSON/HTML monitoring outputs.

6. Script-by-script explanation

Script	When to use it	What it creates
scripts/run_demo.py	Run this first after installing requirements.	Processes sample cases, hashes evidence, creates demo results.
scripts/generate_synthetic_cases.py	Use this to create a bigger fake dataset.	CSV file with synthetic incidents.
scripts/batch_triage.py	Use this to process many cases from a CSV.	Batch triage results CSV.
scripts/monitor_patterns.py	Use this after cases have been processed.	Pattern report in JSON and HTML.
scripts/export_case_packet.py	Use this for one case after triage.	Markdown case packet for human review.

7. API explanation

Endpoint	Purpose
/health	Checks whether the API is running.
/triage	Submits an incident report and receives classification, risk, guidance, and draft complaint.
/evidence/text	Hashes text evidence and logs it in the evidence chain.
/case/{case_id}	Retrieves a stored case.
/patterns	Returns anonymized repeat-pattern monitoring summary.
/feedback	Stores reviewer or support-worker feedback.
/dashboard/summary	Returns metrics for the dashboard.

The dashboard is built with Streamlit. It reads the SQLite database and displays case counts, evidence counts, category distribution, priority distribution, recent cases, repeat pattern notes, and audit events.

8. Evidence vault and chain-of-custody

The project does not store real files by default. Instead, it hashes evidence text using SHA-256. A hash is like a digital fingerprint. If the evidence changes, the hash changes. The system also creates a chain hash by combining the previous evidence chain hash with the new evidence hash. This simulates blockchain-style integrity in a simple local way.

```
previous_chain_hash + case_id + evidence_name + evidence_hash + timestamp -> chain_hash
```

This design is useful for a portfolio because it demonstrates evidence integrity and auditability without requiring real blockchain infrastructure.

9. Monitoring, audit, and privacy

A sensitive system must record what happened without exposing unnecessary personal information. This project logs cases, evidence hashes, feedback, and audit events. Suspect identifiers are hashed before repeat-pattern tracking. This means the system can notice repeated behavior while reducing identity exposure risk.

The monitoring report shows case counts, evidence counts, category distribution, priority distribution, and repeat anonymized patterns. It is not a criminal database. It is a demo of how responsible monitoring could be designed.

10. How to run the project locally

Step 1: create and activate a virtual environment.

```
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
```

Step 2: install requirements.

```
pip install -r requirements.txt
```

Step 3: run the demo.

```
python scripts/run_demo.py
```

Step 4: run the API.

```
PYTHONPATH=src uvicorn cybershield_ai.api.main:app --host 0.0.0.0 --port 8000 --reload
```

Step 5: run the dashboard in a new terminal.

```
PYTHONPATH=src streamlit run dashboards/streamlit_dashboard.py
```

Step 6: run tests.

```
python -m pytest tests -q
```

11. How to upload to GitHub

Recommended method: unzip the project, create a new GitHub repository, then upload all extracted files and folders. Do not upload generated databases, real evidence, real reports, or virtual environment folders.

```
git init
git add .
git commit -m "Initial CyberShield AI project"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/cybershield-ai.git
git push -u origin main
```

Good repository name: cybershield-ai-digital-rights-safety-justice or cybershield-ai.

Good repository description: AI-assisted cyber safety and digital rights support system with victim triage, complaint drafting, evidence hashing, anonymized pattern monitoring, FastAPI, Streamlit, Docker, and CI.

12. How to add it to your resume

Project title:

CyberShield AI: Digital Rights, Safety and Justice Support System

Resume bullets:

- Built a safe AI-assisted cyber incident triage system that classifies victim reports, scores urgency, retrieves support guidance, and generates complaint/FIR-style drafts for human review.
- Implemented a privacy-aware evidence vault using SHA-256 hashing and hash-chain audit logs to demonstrate digital evidence integrity and chain-of-custody concepts.
- Created anonymized repeat-pattern monitoring for cyber harassment, blackmail, hacking, impersonation, doxxing, and financial fraud using hashed identifiers and SQLite audit storage.
- Developed FastAPI endpoints, a Streamlit monitoring dashboard, batch scripts, Docker support, and GitHub Actions CI tests for an end-to-end production-style AI safety prototype.

Skills to list: Python, FastAPI, Streamlit, SQLite, Pydantic, Scikit-learn, TF-IDF Retrieval, RAG-style Search, SHA-256 Hashing, Evidence Integrity, Privacy-aware NLP, Docker, GitHub Actions, MLOps.

13. How to explain this project in an interview

A strong non-technical explanation:

This project is a cyber safety support platform. Many victims of online harassment, hacking, impersonation, blackmail, or digital fraud do not know how to report the incident or preserve evidence properly. My system collects the victim statement, organizes it into a structured case, suggests a category and risk level, retrieves relevant support guidance, drafts a complaint-style document for human review, hashes evidence to protect integrity, and monitors anonymized repeat patterns. I designed it with safety limits: it does not give final legal advice, does not submit cases automatically, and does not expose real identities.

A strong technical explanation:

The backend is built with FastAPI. The core orchestration service calls a redaction utility, keyword-based case classifier, risk scorer, local TF-IDF RAG engine, complaint draft generator, evidence vault, and pattern tracker. SQLite stores cases, evidence metadata, feedback, and audit events. Streamlit provides monitoring. GitHub Actions runs tests. Docker files are included for deployment.

14. How to rebuild a similar project from scratch

Step	Action
1	Define the user problem and safety boundary.
2	Design the folder structure before coding.
3	Create data models for inputs and outputs.
4	Build one simple service at a time: classifier, risk scorer, retriever, draft generator.
5	Add a database for storage and audit logs.
6	Add API endpoints to expose the workflow.
7	Add a dashboard for business/non-technical users.
8	Add scripts for demo, batch processing, monitoring, and export.

Step	Action
9	Add tests and GitHub Actions CI.
10	Write README, architecture, system card, and ethics note.

15. Common errors and fixes

Error	Fix
ModuleNotFoundError: cybershield_ai	Run commands with PYTHONPATH=src or install the package properly.
No module named streamlit	Install full requirements with pip install -r requirements.txt.
Dashboard shows no data	Run python scripts/run_demo.py first.
API starts but triage fails	Check that data/demo/legal_knowledge.md exists and dependencies are installed.
GitHub Actions fails with no tests ran	Make sure tests/ folder exists and ci.yml runs python -m pytest tests -q.

Final note

This project is designed to impress because it is not just a model. It is a full AI product prototype with safety, privacy, auditability, monitoring, documentation, tests, API, dashboard, and deployment files. Those are the parts recruiters look for when they want evidence of real engineering maturity.